



PDF Download
3746252.3761211.pdf
18 January 2026
Total Citations: 0
Total Downloads: 158

Latest updates: <https://dl.acm.org/doi/10.1145/3746252.3761211>

RESEARCH-ARTICLE

Entity-Aware Generative Retrieval for Personalized Contexts

JIHYEONG JEON, Seoul National University, Seoul, South Korea

JIWON LEE, Seoul National University, Seoul, South Korea

CHEOL RYU, Samsung Electronics Co., Ltd., Suwon, Gyeonggi-do, South Korea

U KANG, Seoul National University, Seoul, South Korea

Open Access Support provided by:

Seoul National University

Samsung Electronics Co., Ltd.

Published: 10 November 2025

[Citation in BibTeX format](#)

CIKM '25: The 34th ACM International
Conference on Information and
Knowledge Management
November 10 - 14, 2025
Seoul, Republic of Korea

Conference Sponsors:

SIGWEB
SIGIR

Entity-Aware Generative Retrieval for Personalized Contexts

Jihyeong Jeon
Seoul National University
Seoul, South Korea
jeonjihyeong@snu.ac.kr

Jiwon Lee
Seoul National University
Seoul, South Korea
torijwl77@snu.ac.kr

Cheol Ryu
Samsung Electronics
Seoul, South Korea
cheol.ryu@samsung.com

U Kang
Seoul National University
Seoul, South Korea
ukang@snu.ac.kr

Abstract

Given a user query containing ambiguous and user-specific references, how can we effectively retrieve personalized information? Personalized information retrieval (PIR) requires resolving context-dependent cues — such as nicknames, personal locations, or temporal expressions. This poses challenges for conventional retrievers, including dense and generative models, which often struggle with entity ambiguity and generalization to user-specific contexts.

In this paper, we propose PEARL (Personalized Entity-Aware Generative Retrieval), a novel generative retrieval framework for personalized IR. PEARL addresses key challenges through three components: (i) entity-aware annotation with span-level regularization to reduce lexical sensitivity, (ii) prefix-based contrastive learning to capture structural alignment between lexically divergent query-passage pairs, and (iii) context diversification to improve robustness against user-specific variations.

Empirical results on both an existing PIR dataset and our new large-scale synthetic benchmark PAIR show that PEARL consistently outperforms strong baselines under zero-shot evaluation. Notably, PEARL achieves the state-of-the-art performance in Hits@1 and MRR@10, demonstrating its effectiveness for retrieval in personalized user contexts. Our dataset is available at <https://www.github.com/pearl-pair/pearl>.

CCS Concepts

• **Information systems** → **Retrieval models and ranking**; *Personalization*; Data mining.

Keywords

information retrieval; generative models; personalized retrieval; deep learning; named-entity recognition

ACM Reference Format:

Jihyeong Jeon, Jiwon Lee, Cheol Ryu, and U Kang. 2025. Entity-Aware Generative Retrieval for Personalized Contexts. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25)*, November 10–14, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3746252.3761211>

1 Introduction

How can we effectively retrieve personalized information when user queries contain ambiguous, user-specific references? This question

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CIKM '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2040-6/2025/11

<https://doi.org/10.1145/3746252.3761211>

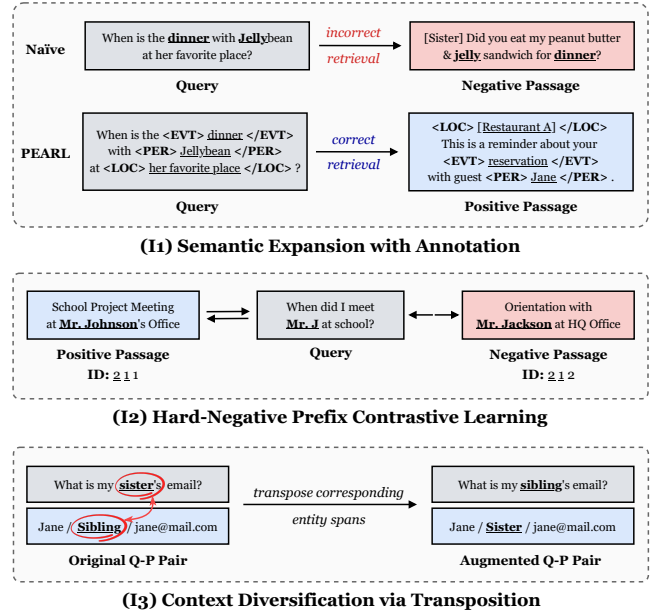


Figure 1: The overview of the proposed method PEARL. (I1) Labels user-specific references with entity tags to improve structural understanding and reduce reliance on surface overlap (e.g., “jelly”, “dinner”). (I2) Encourages discrimination between structurally similar but semantically distinct passages using hierarchical ID prefixes. (I3) Augments data by swapping aligned entity mentions (e.g., “sister” ↔ “sibling”) to enhance robustness to contextual variation.

encapsulates the central challenge of *personalized information retrieval*: retrieving relevant information when queries reference user-specific entities — such as nicknames, recurring locations, or temporally grounded events — that are often absent from general-purpose corpora and hard to resolve with standard retrieval methods.

Personalized information retrieval is increasingly critical in applications such as personal assistants, private search engines, and digital memory systems [4, 14–17, 20, 21]. Yet, it remains underexplored in IR and NLP literature [34], largely due to its inherent complexity. User-specific signals are often sparsely annotated, lexically idiosyncratic, and closely intertwined with contextual semantics, making it difficult to model relevance beyond mere textual similarity [39].

Traditional retrieval paradigms — such as sparse methods [29] or dense retrieval with dual encoders [18, 24] — struggle in this setting, as they primarily rely on lexical overlap or embedding proximity [8]. More recently, generative retrieval methods [32, 33, 35] have emerged as promising alternatives by modeling retrieval as sequence generation. However, these models are still largely agnostic to entity ambiguity and personalization cues. Naïvely

applying generative retrievers to personalized corpora leads to brittle behavior and limited generalization across user contexts.

To address these challenges, we propose PEARL (Personalized Entity-Aware Generative Retrieval), a generative retrieval framework tailored for personalized information retrieval. PEARL introduces three core innovations. First, we reduce lexical sensitivity to ambiguous expressions by applying entity-aware annotations and span-based regularization, allowing the model to focus on semantic roles over surface-level lexical cues. Second, we introduce a prefix-based contrastive loss that encourages alignment between query-passage pairs that are semantically similar yet lexically divergent, while discouraging associations between unrelated pairs. Finally, we improve robustness through context diversification, exposing the model to a broader range of personalized contexts during training. An overview is shown in Figure 1.

Together, these components are trained end-to-end under a unified generative objective, where both queries and passages are represented via structured, semantically grounded identifiers derived from hierarchical k -means clustering. This enables PEARL to retrieve relevant documents despite vague or ambiguous user-specific cues and to generalize across diverse personalized contexts.

To comprehensively evaluate our approach, we introduce and open-source a new large-scale synthetic benchmark, PAIR, generated with LLMs. PAIR comprises over 150 diverse user personas and is designed to simulate on-device scenarios, providing a rich testbed for personalized retrieval. Our empirical results show that PEARL consistently outperforms strong baselines — including sparse retrievers, dual-encoder dense models, and standard generative retrievers — on multiple PIR benchmarks, achieving the state-of-the-art performance in retrieval metrics such as Hits@k and MRR@k.

Our Main Contributions are summarized as follows:

- We propose PEARL, a novel generative retrieval framework designed to handle ambiguous, context-dependent queries in personalized IR scenarios.
- We introduce three key techniques — entity-span regularization, prefix-based contrastive learning, and context diversification — to improve robustness and structural generalization.
- We demonstrate that PEARL outperforms competitive baselines on both public and synthetic PIR datasets, achieving superior performance under zero-shot retrieval conditions.
- We publicly release PAIR (Personalized Ambiguous Information Retrieval), a benchmark dataset reflecting realistic user behaviors on mobile-devices, to support future research on personalized information retrieval.

2 Background

Personalized Information Retrieval. Personalized information retrieval seeks to retrieve relevant passages based on user-specific queries, where queries and passage sets vary across users due to differences in personal references, contexts, and semantic preferences. We give a formal definition of the personalized information retrieval problem as follows:

PROBLEM 1 (PERSONALIZED INFORMATION RETRIEVAL).

Given:

- A set of personalized queries $\mathcal{Q} = \{q_{(i)}\}_{i=1}^{|\mathcal{Q}|}$ containing potentially ambiguous, user-specific references.

- A user-specific corpus of personal passages $\mathcal{P} = \{p_{(i)}\}_{i=1}^{|\mathcal{P}|}$.
- An integer $k \leq |\mathcal{P}|$ specifying the number of passages to retrieve.

Objective: For each query $q \in \mathcal{Q}$, retrieve an ordered subset of top- k passages $\mathcal{P}' = \{p_1, \dots, p_k\}$ from the candidate set $\mathcal{P}$, ranked according to their relevance to the query q , such that the following standard retrieval metrics are maximized:

$$\begin{aligned} \text{Hits@}k &= \frac{1}{|\mathcal{Q}|} \sum_{i=1}^{|\mathcal{Q}|} \mathbf{1}[p_{(i)}^* \in \text{top}_k(q_{(i)})], \\ \text{MRR@}k &= \frac{1}{|\mathcal{Q}|} \sum_{i=1}^{|\mathcal{Q}|} \sum_{p_{(j)} \in \text{top}_k(q_{(i)})} \frac{\mathbf{1}[p_{(j)} = p_{(i)}^*]}{\text{rank}(p_{(j)})}, \end{aligned} \quad (1)$$

where $p_{(i)}^*$ is the ground-truth passage associated with the i^{th} query $q_{(i)}$, $\text{top}_k(q_{(i)})$ denotes the set of the top- k ranked passages retrieved for query $q_{(i)}$, $\text{rank}(p_{(j)})$ is the index of the passage $p_{(j)}$ within the ordered-retrieved set, and $\mathbf{1}[\cdot]$ denotes the indicator function.

Zero-Shot Generalization Constraint. We assume a constrained-resource retrieval scenario typical for mobile deployment, in which the retrieval model must generalize effectively to diverse users without post-deployment training or fine-tuning. Formally, let $\mathcal{Q}_{\text{train}}$ and $\mathcal{P}_{\text{train}}$ denote the training sets of queries and passages. For the evaluation queries and passages $(q, p^*) \in \mathcal{Q}_{\text{eval}} \times \mathcal{P}_{\text{eval}}$, it must hold that:

$$\mathcal{Q}_{\text{train}} \cap \mathcal{Q}_{\text{eval}} = \emptyset, \quad \mathcal{P}_{\text{train}} \cap \mathcal{P}_{\text{eval}} = \emptyset. \quad (2)$$

This ensures a strict zero-shot retrieval scenario, where evaluation pairs (q, p^*) are completely unseen during model training.

Generative Information Retrieval. In recent years, generative retrieval [7, 28, 33] has emerged as a compelling alternative to traditional information retrieval paradigms [12, 19], such as sparse retrieval — e.g., BM25 [29] — and dense retrieval — e.g., dual-encoder models [18, 24]. Unlike these retrieval and rank methods, generative retrieval treats the retrieval task as a sequence generation problem, where the model is trained to directly generate passage identifiers (PIDs) corresponding to relevant passage given a query.

Formally, let $\mathcal{Q} = \{q_{(1)}, \dots, q_{(N)}\}$ denote a set of queries, and $\mathcal{P} = \{p_{(1)}, \dots, p_{(N)}\}$ be the corresponding set of passages. In generative retrieval, each query $q_{(i)}$ and its corresponding passage $p_{(i)}$ are assigned a common *token sequence identifier* $y_{(i)} = \{y_{i,1}, \dots, y_{i,L_i}\}$, which serves as their passage identifier (PID).

The process of assigning passage identifiers — referred to as passage tokenization — is designed to yield *semantically structured* identifiers, which embed content-aware information to support downstream retrieval and generation tasks. Specifically, a pre-trained encoder $f(\cdot)$ maps each passage to a dense representation $f(p_{(i)})$, which is then clustered using a semantic algorithm such as hierarchical k -means. Each passage is assigned a token sequence $y_{(i)}$ that corresponds to its path in the cluster hierarchy, resulting in a passage identifier that encodes meaningful semantic information.

At training time, the model is given a query $q_{(i)}$ or a passage $p_{(i)}$ and trained to generate the corresponding passage identifier $y_{(i)}$, such that the passage $p_{(i)}$ retrieved by $y_{(i)}$ is relevant to $q_{(i)}$. This is modeled with a sequence-to-sequence architecture $\mathcal{F}_\theta$, parameterized by θ , trained to minimize the negative log-likelihood:

$$\mathcal{L}_0 = - \sum_{i=1}^N \sum_{t=1}^{L_i} \log p_\theta(y_{i,t} \mid y_{i,<t}, q_{(i)}) p_\theta(y_{i,t} \mid y_{i,<t}, p_{(i)}), \quad (3)$$

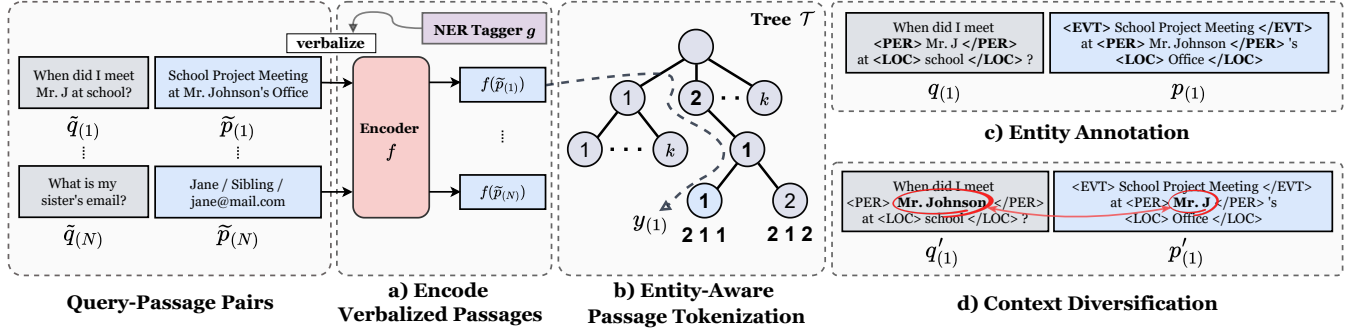


Figure 2: Overview of the preprocessing pipeline in PEARL. (a) Each passage $\tilde{p}_{(i)}$ is verbalized and then encoded by a pre-trained encoder $f(\cdot)$. (b) The resulting embeddings are clustered via hierarchical k -means to construct a semantic prefix-tree $\mathcal{T}$, from which structured passage identifiers $y_{(i)}$ are derived. (c) Queries and passages are annotated with special entity tokens to reduce lexical ambiguity. (d) To improve robustness, transpositional augmentation is applied by swapping entity spans across aligned query-passage pairs, generating diversified examples $(q'_{(i)}, p'_{(i)})$. This preprocessing facilitates structured, entity-aware learning for robust personalized retrieval.

where, at each decoding step t , the model is conditioned on the tokenized query and the tokenized prefix of the target sequence, $y_{i,t} = \{y_{i,1}, \dots, y_{i,t-1}\}$, to predict the next token $y_{i,t}$. This corresponds to the standard autoregressive cross-entropy loss used in sequence generation, optimized with teacher forcing. At inference time, given a new query q , the model autoregressively generates the most likely identifier $\hat{y} = \mathcal{F}_{\theta}(q)$, which is then used to retrieve the corresponding passage by mapping $\hat{y}$ to its associated p .

We adopt the generative retrieval paradigm for its ability to model structured, semantically disentangled representations of user intent. Personalized queries often involve context-specific entities — such as people, events, locations, and temporal expressions — with rich compositional structure. Generative retrieval accommodates this by producing hierarchical token sequences as passage identifiers, reflecting semantic paths (e.g., person $\rightarrow$ event $\rightarrow$ location) rather than mapping queries to a flat embedding space.

This inductive bias enables finer disambiguation and robust generalization to user-specific references. In contrast to dense retrieval’s continuous similarity scoring, generative retrieval facilitates prefix-conditioned decoding, symbolic constraints, and interpretable alignment between query semantics and identifier structure. These properties are especially advantageous in PIR, where retrieval relevance depends not only on lexical matching but also on understanding implicit context and entity interactions.

3 Proposed Method

In this section, we propose PEARL, a novel entity-aware generative retrieval framework for personalized information retrieval. Our approach addresses the following key challenges:

- C1 Lexical Disambiguation.** User-specific ambiguities arising from personalized contexts often lead to the retrieval of semantically irrelevant passages. *How can we retrieve the correct (gold) passage without relying on explicit lexical disambiguation of obscure terms grounded in personal context?*
- C2 Discerning Semantic Structure.** Standard cross-entropy objectives struggle to distinguish between passages that are structurally similar but semantically distinct — particularly when similar entity types co-occur in comparable contexts. *How can*

we train models to differentiate semantic intent despite structural overlap, capturing both similarity and semantic divergence?

- C3 Context-Invariant Representation.** Semantically equivalent queries or passages can appear in diverse surface forms due to paraphrastic variation in how meaning is framed or embedded in context, which may lead the model to infer divergent meanings. *How can we train the model to yield consistent representations for inputs that convey the same meaning despite syntactic and contextual variation?*

To address these challenges, PEARL unifies the following three core ideas into an end-to-end generative retrieval framework for accurate personalized information retrieval:

- I1 Semantic Expansion with Annotation (§3.1).** We enrich both queries and passages by annotating ambiguous or user-specific references with entity labels using a pre-trained NER model. These annotations enable the model to access higher-level structural signals beyond surface-level semantics. To fully leverage this abstraction, we introduce entity-aware passage tokenization and entity-span mass regularization, which jointly guide the model toward learning more robust and disambiguated semantic representations.
- I2 Hard-Negative Prefix Contrastive Learning (§3.2).** We propose a contrastive training objective using hard negatives mined from a prefix-tree built via hierarchical semantic clustering. We also compute similarity scores with entity-structural embeddings that jointly encode high-level sentence structure and semantics, enabling the model to discern subtle semantic differences even among structurally similar candidates.
- I3 Context Diversification via Transposition (§3.3).** We introduce a data augmentation strategy that swaps entities between queries and passages with matching entity types and high semantic relevance. These transposed pairs retain core meaning but alter surrounding context, encouraging the model to learn context-invariant representations for semantically equivalent yet lexically diverse inputs.

Figures 2 and 3 summarizes how PEARL integrates these components into a cohesive framework.

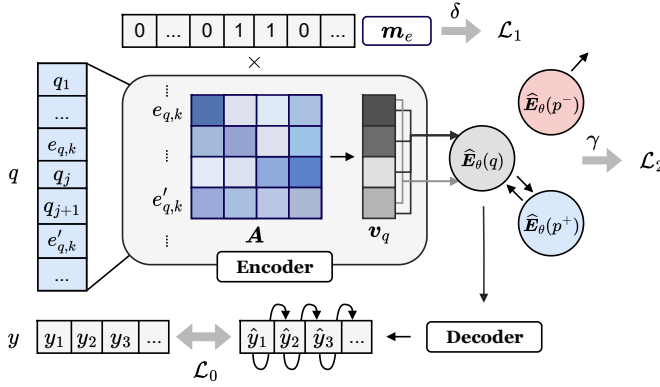


Figure 3: The overall process of PEARL.

Given an annotated query q , the encoder produces token representations v_q and a self-attention matrix A ; the decoder then generates a structured identifier sequence $\hat{y}_{(i)} = \{\hat{y}_{i,1}, \hat{y}_{i,2}, \dots\}$, which is used to retrieve the corresponding passage. PEARL is trained with three objectives:

- **Generation loss $\mathcal{L}_0$** is a standard cross-entropy objective applied during sequence generation, constrained by a prefix-tree to ensure valid and syntactically well-formed identifiers.
- **Entity-span mass regularization $\mathcal{L}_1$** reduces over-reliance on ambiguous or user-specific tokens by penalizing excessive attention mass allocated to entity spans. This is computed using the attention matrix A and entity mask m_e , with threshold δ to control the penalty.
- **Contrastive loss $\mathcal{L}_2$** encourages alignment between the query embedding $\hat{E}_\theta(q)$ and its corresponding positive passage $\hat{E}_\theta(p^+)$, while pushing away a semantically similar hard negative $\hat{E}_\theta(p^-)$. Embeddings are composed of token representations weighted by entity-aware scaling factor γ , and the temperature τ controls the sharpness of similarity discrimination.

3.1 Semantic Expansion with Annotation

Retrieval models frequently encounter difficulties when interpreting user-specific terms embedded in personalized contexts, often due to lexical ambiguity and insufficient structural cues. To address this challenge, we propose a structured, entity-aware pipeline that enhances semantic disambiguation and improves robustness across personalized inputs. During preprocessing, we first identify ambiguous or user-specific mentions in both queries and passages using a pre-trained named entity recognition model $g(\cdot)$ [3]. These detected entity spans serve two key roles: (1) enabling the semantic structuring of passage identifiers, and (2) serving as anchor points for regulating model attention during training. The former is realized through an *entity-aware passage tokenization* strategy based on hierarchical semantic clustering, which systematically encodes passage-level entity semantics into structured identifiers. The latter is an *entity-span mass regularization* mechanism, designed to regulate attention allocation toward annotated entity spans, thereby promoting robust and generalizable semantic representations less reliant on ambiguous entities.

Entity-Aware Passage Tokenization. To construct passage identifiers that reflect hierarchical semantic structure, we perform

entity-aware passage tokenization via semantic clustering. Specifically, this process incorporates entity-level context by enriching each passage with natural language descriptions of identified entities — a process known as verbalization. For example, the raw passage

School Project Meeting at Mr. Johnson’s Office
is transformed into:

[School Project Meeting is an event] at
[Mr. Johnson is a person]’s [Office is a location]

This verbalization enables frozen pre-trained language models, such as bert-base-uncased [10], to capture entity semantics without additional fine-tuning or entity token embeddings. Verbalized passages are then embedded and clustered using recursive k -means to form a hierarchical structure over the corpus. The resulting identifiers serve as generation targets during training and guide structured query-to-passages alignment in personalized retrieval. This process is illustrated in panels a) and b) of Figure 2.

Entity Annotation. To prepare for training, we insert special entity tokens around detected entity spans in both queries and passages, as illustrated in panel c) of Figure 2. Given a query-passage pair $(\tilde{q}, \tilde{p})$, the annotated sequences take the form:

$$\begin{aligned} \tilde{q} = \{q_1, \dots, q_m\} &\rightarrow q = \{e_{q,1}, q_1, \dots, q_a, e'_{q,1}, \dots, q_m\}, \\ \tilde{p} = \{p_1, \dots, p_n\} &\rightarrow p = \{p_1, \dots, e_{p,k}, p_3, \dots, p_d, e'_{p,k}, \dots, p_n\}, \end{aligned} \quad (4)$$

where each $e_{\cdot}$ represents a special entity token indicating the start position of an entity span (e.g., <PER>, <LOC>, <EVT>, or <TIM>), and each $e'_{\cdot}$ indicates the corresponding end position (e.g., </PER>, </LOC>, </EVT>, or </TIM>). The spans $\{q_1, \dots, q_a\}$ and $\{p_3, \dots, p_d\}$ correspond to entity mentions in the query and passage. For example, the raw query

When did I last meet Mr. J at school?

is transformed into:

When did I last meet <PER> Mr. J </PER>
at <LOC> school </LOC> ?

Entity annotations in the queries are used for entity-span mass regularization, while annotations in both queries and passages support context diversification via entity transposition in Section 3.3.

Entity-Span Mass Regularization. To promote robust generalization and reduce over-reliance on ambiguous or user-specific entity spans, we introduce a regularization term based on attention mass assigned to entity span tokens. Formally, let $A \in \mathbb{R}^{L \times L}$ denote the self-attention matrix, averaged over heads and layers, for an encoded token sequence of length L , and let $m_e \in \{0, 1\}^L$ be a binary mask that assigns 1 to tokens within entity spans and 0 elsewhere. Here, $A_{i,j}$ denotes the attention weight from the i -th token to the j -th token in the input sequence.

We compute the entity-span mass β with the attention weights allocated to entity spans as:

$$\beta = \frac{\sum_{i,j=1}^L A_{i,j} m_{e_j}}{n_e \cdot \sum_{i,j=1}^L A_{i,j}} \quad (5)$$

where $n_e = \sum_{i=1}^L m_{e_i}$ is the number of tokens in the entity spans. Our entity-span mass regularization loss is then formulated as:

$$\mathcal{L}_1 = -\log(1 - \max(\beta - \delta, 0) + \epsilon), \quad (6)$$

Algorithm 1 Prefix-Tree Construction

Input: A set of training identifiers $\mathcal{Y} = \{y^{(i)} = [y_1^{(i)}, \dots, y_{L_i}^{(i)}]\}_{i=1}^{|\mathcal{Y}|}$
Output: A prefix-tree $\mathcal{T}$ storing all identifier paths

```

1:  $\mathcal{T}.root \leftarrow \text{NODE}()$   $\triangleright$  initialize root node with empty value
2: for all  $y = [y_1, \dots, y_L] \in \mathcal{Y}$  do
3:    $v \leftarrow \mathcal{T}.root$   $\triangleright$  start from the root
4:   for  $j = 1$  to  $L$  do
5:     if  $\text{FINDCHILD}(v, y_j)$  is None then  $\triangleright$  find child with value  $y_j$ 
6:        $\text{ADDCHILD}(v, \text{NODE}(y_j))$   $\triangleright$  add child node with value  $y_j$ 
7:        $v \leftarrow \text{FINDCHILD}(v, y_j)$   $\triangleright$  traverse to child with value  $y_j$ 
8: return  $\mathcal{T}$ 

```

where $\delta \in [0, 1]$ is a predefined threshold controlling the acceptable entity-span mass and $\epsilon > 0$ is a constant for numerical stability.

For instance, in the query "When did I last meet Mr. J?", the span "Mr. J" may receive disproportionately high attention due to its semantic ambiguity. To mitigate this, the regularization loss $\mathcal{L}_1$ penalizes this over-attention when $\beta > \delta$, encouraging the model to balance attention between entity-specific and general content. This leads to more robust representations, particularly in personalized settings where entity references may vary across users.

3.2 Hard-Negative Prefix Contrastive Learning

Standard cross-entropy training struggles to differentiate subtle semantic intent, particularly among passages exhibiting structural similarity or overlapping entity patterns. To address this, we integrate a structured contrastive learning objective that samples hard negatives from a semantically hierarchical prefix-tree built from entity-aware passage identifiers. Furthermore, we incorporate entity-structural embeddings into similarity calculations, ensuring the learned embeddings capture both structural information and subtle semantic differences.

For instance, as shown in (I2) of Figure 1, given the query "When did I meet Mr. J at school?" and its positive passage "School Project Meeting at Mr. Johnson's Office," a suitable hard-negative would be "Orientation with Mr. Jackson at HQ Office." Despite similar surface structure (e.g., person and location entities), the latter differs in contextual relevance, making it an effective contrastive example.

Formally, given a positive passage p^+ with identifier y^+ , we aim to sample a hard-negative p^- whose identifier y^- shares a common prefix with y^+ but diverges at some later depth. This is achieved efficiently via depth-first traversal over a prefix-tree $\mathcal{T}$, which encodes identifiers as paths from the root to leaf nodes, where each node holds a single token value.

As shown in Algorithm 1, the prefix-tree is constructed by iterating over training identifiers and incrementally adding child nodes that store token values. At each level, the function FINDCHILD returns the child node of the current node v whose value matches the current token y_j . If such a node does not exist, a new node $\text{NODE}(y_j)$ — which initializes a node with its value set to y_j — is created and added as a child of v using ADDCHILD . Then, traversal proceeds to the child. This process results in a compact hierarchical representation of all identifiers that supports efficient prefix-based access.

Algorithm 2 describes the hard-negative sampling process given a query identifier y^+ . The algorithm first descends the tree along the path defined by y^+ , pushing (y_j^+, v) pairs onto a stack and the

Algorithm 2 Hard-Negative Sampling from the Prefix-Tree

Input: Prefix-tree $\mathcal{T}$, positive identifier $y^+ = [y_1^+, \dots, y_L^+]$
Output: Hard-negative identifier y^-

```

1:  $stack \leftarrow []$   $\triangleright$  path memory
2:  $y^- \leftarrow []$   $\triangleright$  initialize hard-negative identifier as a stack
3:  $v \leftarrow \mathcal{T}.root$ 
4: for  $j = 1$  to  $L$  do  $\triangleright$  descend along  $y^+$  path
5:    $\text{push}(y_j^+, v)$  onto  $stack$ 
6:    $\text{push } y_j^+$  onto  $y^-$ 
7:    $v \leftarrow \text{FINDCHILD}(v, y_j^+)$ 
8: for  $depth = |stack| - 1$  downto 0 do  $\triangleright$  climb up path
9:    $(tok, parent) \leftarrow \text{POP}(stack)$ 
10:   $\text{POP}(y^-)$   $\triangleright$  remove last token for replacement
11:   $siblings \leftarrow \{c \in \text{CHILDREN}(parent) \mid c.VALUE \neq tok\}$ 
12:  if  $siblings \neq \emptyset$  then
13:     $v \leftarrow \text{RANDOMCHOICE}(siblings)$   $\triangleright$  choose a sibling node
14:     $\text{push } v.VALUE$  onto  $y^-$   $\triangleright$  add value of visited node to  $y^-$ 
15:    while  $\text{CHILDREN}(v) \neq \emptyset$  do  $\triangleright$  DFS toward leaf
16:       $v \leftarrow \text{RANDOMCHOICE}(\text{CHILDREN}(v))$ 
17:       $\text{push } v.VALUE$  onto  $y^-$ 
18:  return  $y^-$ 

```

tokens y_j^+ onto y^- to initialize the candidate. It then backtracks from the deepest node, removing the last token from y^- at each level, to search for a sibling node — i.e., a node with a different value from y_j^+ at the same depth. Once such a sibling is found, the algorithm performs a randomized depth-first walk from that node toward a leaf, appending the value of each visited node to y^- along the way. The algorithm terminates when a leaf is reached, at which point the constructed y^- is returned as the hard-negative identifier.

For the entity-structural embedding, let $E_\theta(x) = \frac{1}{|x|} \sum_{i=1}^{|x|} v_{x_i}$ denote an aggregated embedding of the input sequence x , where $E_\theta(\cdot)$ is an encoder and $v_{x_i} \in \mathbb{R}^d$ is a d -dimensional output embedding vector of individual tokens from the encoder. To effectively capture structural similarities, we explicitly integrate entity-structure information into these embeddings by enhancing them with special entity token embeddings:

$$\widehat{E}_\theta(x) = E_\theta(x) + \gamma \frac{1}{|\mathcal{I}_e|} \sum_{i \in \mathcal{I}_e} v_{x_i}, \quad (7)$$

where $\mathcal{I}_e$ denotes the indices of special entity tokens within the input x ; these tokens — such as <PER>, <LOC>, <EVT>, and <TIM> — are introduced during preprocessing to mark identified entity spans. The scalar γ is a hyperparameter controlling the weight of entity-structure information.

Subsequently, let $\widehat{E}_\theta(q)$, $\widehat{E}_\theta(p^+)$, and $\widehat{E}_\theta(p^-)$ denote the structurally enriched embeddings of a query, its corresponding positive passage, and a sampled hard-negative passage, respectively. Using these, our hard-negative prefix contrastive loss (PCL) resembles the in-batch InfoNCE [18, 26] formulation:

$$\mathcal{L}_2 = -\log \frac{e^{\widehat{E}_\theta(q)^\top \widehat{E}_\theta(p^+)/\tau}}{\sum_{j \in \mathcal{B}} \left(e^{\widehat{E}_\theta(q)^\top \widehat{E}_\theta(p_{(j)}^+)/\tau} + e^{\widehat{E}_\theta(q)^\top \widehat{E}_\theta(p_{(j)}^-)/\tau} \right)}, \quad (8)$$

where $\mathcal{B}$ denotes the training batch, j indexes over the paired positive and negative passages corresponding to queries in the batch, and the temperature parameter τ controls the sharpness of the distribution. This loss encourages the model to distinguish

structurally similar but semantically divergent passages, thereby improving robustness and retrieval accuracy.

3.3 Context Diversification via Transposition

Models trained solely on original data tend to struggle with generalizing across diverse yet semantically equivalent contexts, particularly in personalized retrieval settings. To alleviate this, we propose a data augmentation strategy that systematically swaps entity mentions between queries and passages, conditioned on entity type alignment and semantic relevance. By generating contextually diversified yet semantically consistent training pairs, our method encourages the model to produce invariant representations robust to paraphrastic and contextual variations commonly observed in real-world personalized queries [6]. This process is illustrated in panel d) of Figure 2.

Let $\mathcal{E}(x) = \{\langle t(e_{x,i}), s(e_{x,i}) \rangle\}_{i=1}^k$ be the set of all entity type and the entity spans in the sequence, where $t(e_{x,i})$ is the entity type (e.g., <PER>), and $s(e_{x,i})$ denotes the token indices covered in between special entity tokens $e_{x,i}$ and $e'_{x,i}$. Then, to systematically determine which entity spans to transpose in query-passage pair (q, p) , we utilize a pre-trained reranker model $\mathcal{R}(\cdot)$, capable of producing a self-attention score matrix $S(q, p) \in \mathbb{R}^{(m+n) \times (m+n)}$ between concatenated token sequences $[q; p]$, where m and n are the length of the query and the passage, respectively.

Formally, for each entity span of the query, we calculate a relevance score ρ toward each entity span of the same type — i.e., $t(e_{q,i}) = t(e_{p,j})$ — in the passage:

$$\rho(e_{q,i}, e_{p,j}) = \sum_{u \in s(e_{q,i})} \sum_{v \in s(e_{p,j})} S_{u,m+v}(q, p). \quad (9)$$

If this relevance score exceeds a threshold θ , the corresponding pair $(e_{q,i}, e_{p,j})$ becomes a candidate for transposition. For each query entity span $e_{q,i}$, we select the highest-scoring passage entity span $e_{p,j}^*$ among candidates to perform the transposition, generating augmented sequences:

$$\begin{aligned} q' &= q \left[\text{replace}(s(e_{q,i}) \rightarrow s(e_{p,j}^*)) \right], \\ p' &= p \left[\text{replace}(s(e_{p,j}^*) \rightarrow s(e_{q,i})) \right]. \end{aligned} \quad (10)$$

Specifically, let $s(e_{q,i}) = q_a, \dots, q_b$ and $s(e_{p,j}^*) = p_c, \dots, p_d$, then the augmented query and passage become:

$$\begin{aligned} q' &= \{\dots, e_{q,i}, p_c, \dots, p_d, e'_{q,i}, \dots\}, \\ p' &= \{\dots, e_{p,j}^*, q_a, \dots, q_b, e'_{p,j}, \dots\}. \end{aligned} \quad (11)$$

This procedure is repeated for each query entity span independently, resulting in up to $|\mathcal{E}(q)|$ augmented pairs from each original (q, p) pair, where $|\mathcal{E}(q)|$ denotes the set of annotated entity spans in query q . Note that each augmented query-passage pair (q', p') retains the same identifier y as its original counterpart (q, p) .

By training on these augmented pairs, the model is explicitly guided to associate semantically equivalent contexts, despite lexical variations introduced by entity transposition. Consequently, the model learns context-invariant representations, improving retrieval accuracy and generalization capability during inference.

3.4 Training and Optimization

Training. Our training process follows the standard generative retrieval framework. We begin by assigning structured identifiers to each query-passage pair $(q, p) \in \mathcal{Q}_{\text{train}} \times \mathcal{P}_{\text{train}}$ using entity-aware semantic clustering (Section 3.1). To increase coverage and robustness, we augment the training set with additional pairs $(q', p') \in \mathcal{Q}_{\text{aug}} \times \mathcal{P}_{\text{aug}}$ via context diversification. We also incorporate synthetic queries generated by a fine-tuned DocT5Query [23] model to expose the model to a broader range of natural query formulations. The final training objective combines three loss terms:

$$\mathcal{L} = \mathcal{L}_0 + \lambda_1 \mathcal{L}_1 + \lambda_2 \mathcal{L}_2, \quad (12)$$

where $\mathcal{L}_0$ is the standard sequence-to-sequence cross-entropy loss in Eq. (3) and the auxiliary losses $\mathcal{L}_1$ and $\mathcal{L}_2$, scaled by hyperparameters λ_1 and λ_2 , correspond to the objectives in Sections 3.1 and 3.2, respectively.

Inference. At inference time, we generate the most probable document identifier using autoregressive decoding with beam search. Following prior works [33, 35], we apply prefix-tree constrained decoding, where generation is restricted to tokens that correspond to valid continuations within the prefix tree $\mathcal{T}$ constructed using Algorithm 1, at each step of the autoregressive process.

For zero-shot retrieval, we generate identifiers for unseen passages and check whether a query produces the same ID. To encourage one-to-one mapping, we use a large beam size during identifier generation, as identical identifiers can occasionally be assigned to multiple passages. Post-processing includes temporal filtering for queries with explicit time expressions and reranking with a pre-trained model to refine alignment. These steps are applied consistently across all baselines for fair comparison.

4 Experiments

We address the following research questions through experiments:

RQ1 Zero-shot Passage Retrieval (§4.2). Does PEARL outperform existing sparse, dense, and generative baselines in retrieving personalized passages under zero-shot settings?

RQ2 Ablation Studies (§4.3). How does each proposed component — Entity-Mass Regularization (EMR), Prefix Contrastive Loss (PCL), and Context Diversification via Transposition (CDT) — contribute to retrieval performance?

RQ3 Hyperparameter Sensitivity Analysis (§4.4). How robust is PEARL to changes in key hyperparameters associated with its loss functions and embedding strategies?

4.1 Experimental Setup

Dataset. To comprehensively evaluate personalized information retrieval (PIR), we use the publicly available LiHuaWorld dataset from MiniRAG [11] and introduce a new large-scale benchmark, PAIR (Personalized Ambiguous Information Retrieval), specifically curated to reflect the challenges of user-dependent retrieval. Unlike existing IR datasets that focus on general-purpose factoid queries (e.g., “What is the capital of France?”) and lack the ambiguity intrinsic to personalized corpora, PAIR centers on user-authored data. This includes private messages, schedules, and contacts, where meaning is inherently tied to the user’s identity, routines, and personal references.

Table 1: Example query-passage pairs $(\tilde{q}, \tilde{p}) \in Q \times \mathcal{P}$ from the proposed PAIR dataset. Bolded spans indicate entity mentions — e.g., people, locations, or events — which can be user-dependent and ambiguous without further specification.

Persona ID	Query $\tilde{q}$	Gold Passage $\tilde{p}$
132	What is the date of my last doctor visit?	Annual Health Check-Up Comprehensive physical exam with Dr. Patel
111	Did Mom ever mention when she's coming for a visit?	Esther I'll be in town for a couple of days starting August 10th.
111	Where did I attend the tech conference this summer?	Tech Innovators Summit Three-day event focusing on AI advancements and ethical considerations
104	Who is the person I had a late-night chat with about tattoo designs?	Tess Hey! What do you think about adding some tribal patterns?
112	What did Sweetie tell me about our dinner plans for this week?	Charlie Let's meet at The Rustic Table for dinner this Friday. They have a great menu!

Table 2: Statistics of the personalized information retrieval datasets used in our experiments. $\mathcal{D} = Q \times \mathcal{P}$ denotes the set of query-passage pairs, and N^P represents the number of distinct personas in the training and evaluation sets.

Dataset	$ \mathcal{D}_{train} \cup \mathcal{D}_{eval} $	$ \mathcal{D}_{train} $	N^P_{train}	$ \mathcal{D}_{eval} $	N^P_{eval}
LihuaWorld [11]	512	-	-	512	1
PAIR	51,522	47,391	149	4,131	12

To construct PAIR, we employ GPT-4o [1] to generate richly detailed fictional personas, each defining a coherent user profile with occupations, hobbies, relationships, frequently visited locations, and recurring events. Using these personas as context, we generate query-passage pairs with Qwen2.5-14B-Instruct [38], ensuring contextual grounding and personalization. As shown in Table 2, PAIR comprises over 51K annotated examples spanning 149 training personas and 12 evaluation personas. However, due to the limited number of data points in the LiHuaWorld dataset, we use it only for evaluating the generalizability of models trained on PAIR.

PAIR is designed to capture two core properties of PIR. First, it provides diverse user contexts at scale: each query-passage pair aligns with a specific persona and includes personalized references (e.g., *Mom*, *Sweetie*, or *Jellybean*) that require contextual disambiguation. Second, it reflects realistic ambiguity, with many queries underspecified in isolation and reliant on implicit entity understanding. Table 1 presents representative examples.

Together, these characteristics make PAIR a challenging and scalable testbed for evaluating personalized retrieval systems, where success hinges on resolving ambiguity, handling implicit references, and adapting to user-specific semantic structures.

Evaluation Metrics. We evaluate model performance using standard retrieval metrics: Hits@ $k \in \{1, 5, 10\}$ and Mean Reciprocal Rank at 10 (MRR@10). Hits@ k measures the proportion of queries where the target passage appears in the top- k retrieved results, while MRR@10 captures the average reciprocal rank of the target within the top 10, emphasizing early retrieval. Together, these metrics offer a comprehensive view of the model's accuracy and ranking quality.

Baselines. We compare PEARL against a broad set of sparse, dense, and generative retrieval methods commonly used in information retrieval. For sparse retrieval, we consider BM25 [29] as a classical term-matching baseline and DocT5Query [23], which enhances document representations by generating synthetic queries using a pre-trained T5 model. For dense retrieval, we include DPR-BM25 [18], a DPR model with hard-negative samples drawn from BM25, Sentence-T5 [24], and GTR-Large [25], both of which encode

queries and passages into a shared embedding space for retrieval via similarity scoring. We also evaluate a range of recent generative retrieval methods. These include DSI [33] and DSI-QG [40], which learn to map queries to document identifiers via sequence generation, and their reranker-augmented variants (e.g., DSI+BGE and DSI-QG+BGE) that improve ranking accuracy with BGE-Reranker [37]. We additionally compare to NCI [35] and its reranker-enhanced variant, which incorporate query augmentation, and MINDER [22], which uses multiview identifiers for improved robustness. Finally, GenRet [31] represents a strong autoregressive retriever with end-to-end learned identifiers. These baselines serve as competitive points of reference across retrieval paradigms, allowing us to evaluate the effectiveness of our entity-aware, context-sensitive design in personalized retrieval settings.

Model Architecture. PEARL is initialized from the standard T5-base [27] architecture. Training is performed on 4× RTX 4090 GPUs (24GB) for up to 25K steps with a mini-batch size of 32 and a learning rate 5e−5, selecting the best checkpoint based on validation retrieval performance. For passage tokenization, we use bert-base-uncased [10] to perform hierarchical k -means clustering with $k = 20$ and the number of leaf nodes is fixed to 20. Post-retrieval re-ranking is performed using bge-reranker-v2-m3 model [36, 37]. For the auxiliary objectives, we set $\delta = 5e-3$ and $\lambda_1 = 1.0$ for entity-mass regularization, and $\tau = 5e-2$, $\gamma = 0.5$, and $\lambda_2 = 1.0$ for prefix contrastive loss. The transposition threshold is fixed at $\theta = 5e-2$. For all generative retrieval methods, we fix the beam size to 100 during both indexing and retrieval. We generate approximately 57K augmented samples from the original training pairs through entity-level context diversification. For entity annotation, we fine-tune the ner-english-fast from the FLAIR framework [2] using 12K annotated examples.

4.2 Zero-shot Passage Retrieval (RQ1)

To evaluate the effectiveness of our framework in generalizing to unseen query-passage pairs, we conduct zero-shot retrieval experiments on two personalized retrieval datasets: PAIR and LiHuaWorld [11]. This setting assesses the model's ability to retrieve the correct passage for a query without having seen the query-passage pair during training, which is particularly challenging when queries contain ambiguous, user-specific references.

Table 3 reports retrieval accuracy in terms of Hits@ $k \in \{1, 5, 10\}$ and MRR@10. As a result, PEARL consistently outperforms all baselines across both datasets and all evaluation metrics. On the PAIR dataset, our method achieves the best performance with 61.15

Table 3: Zero-shot retrieval performance comparison on the PAIR and LiHuaWorld datasets. We report Hits@ $k \in \{1, 5, 10\}$ and MRR@10 for a variety of baseline methods, including sparse, dense, and generative retrievers. PEARL achieves the highest performance across all metrics on both datasets, demonstrating its effectiveness in personalized retrieval under ambiguous and user-specific contexts. Reranker-enhanced variants are denoted with “+BGE-Reranker”.

Method	Params	PAIR				LiHuaWorld [11]			
		Hits@1	Hits@5	Hits@10	MRR@10	Hits@1	Hits@5	Hits@10	MRR@10
<i>Sparse retrieval</i>									
BM25 [29]	-	8.13	19.58	26.31	13.17	31.65	49.51	56.70	39.51
DocT5Query [23]	223M	28.64	38.33	50.11	39.42	34.71	53.63	51.77	37.65
<i>Dense retrieval</i>									
DPR-BM25 [18]	221M	49.73	68.53	79.06	52.07	39.35	52.86	60.95	39.45
Sentence-T5 [24]	335M	51.40	68.60	80.50	61.55	42.47	58.60	60.76	50.88
GTR-Large [25]	335M	52.09	74.36	81.21	65.82	46.58	59.12	61.21	50.49
<i>Generative retrieval</i>									
DSI+BGE-Reranker [33]	818M	48.96	59.75	61.41	53.55	42.16	46.25	53.19	44.63
DSI-QG [40]	223M	51.32	61.48	72.20	56.11	32.65	41.24	49.56	38.99
DSI-QG+BGE-Reranker	791M	59.44	74.19	78.97	65.38	47.87	57.59	59.01	49.84
NCI [35]	308M	52.05	63.91	76.17	58.23	44.44	54.30	57.64	47.26
NCI+BGE-Reranker	876M	60.03	76.83	80.65	67.43	48.90	59.68	61.22	50.03
MINDER [22]	406M	54.62	65.11	70.04	57.73	45.10	54.93	65.96	52.49
GenRet [31]	860M	57.41	72.92	78.44	63.20	49.34	60.71	62.44	54.51
PEARL (proposed)	855M	61.15	77.85	81.46	68.37	50.19	61.87	63.23	55.31

Hits@1 and 68.37 MRR@10, surpassing the strongest reranked baseline (NCI+BGE-Reranker) by +1.12 in Hits@1 and +0.94 in MRR@10. This demonstrates the model’s strong capability to resolve entity ambiguity and structurally align query semantics under zero-shot conditions. Similarly, on LiHuaWorld, which features more diverse and noisy user contexts, PEARL achieves the best performance with 50.19 Hits@1 and 55.31 MRR@10, again outperforming all baseline configurations, including reranker-augmented methods.

Notably, sparse methods such as BM25 and DocT5Query struggle in both domains, with performance significantly lower than their dense and generative counterparts. While dense dual-encoder models like GTR-Large and DPR-BM25 show improved accuracy, their performance is still inferior to generative approaches, suggesting that fixed embedding-based similarity is insufficient for personalized retrieval, especially under high contextual ambiguity.

In general, generative retrievers outperform sparse and dense baselines due to their capacity to model sequence-level semantics. Among them, reranker-augmented variants (e.g., DSI-QG+BGE and NCI+BGE) improve performance, but they still lag behind PEARL. This highlights the unique advantage of our integrated framework, which jointly leverages entity-aware tokenization, contrastive learning, and context diversification, rather than relying on pipeline-based post hoc reranking. Overall, these results validate that PEARL not only achieves state-of-the-art performance but also provides a more robust and principled approach to personalized information retrieval in zero-shot settings.

4.3 Ablation Studies (RQ2)

To assess the individual contribution of each core component in PEARL, we perform an ablation study on both the PAIR and LiHuaWorld datasets. Table 4 reports Hits@ k and MRR@10 under

different configurations by selectively disabling one or more components: Entity-Mass Regularization (EMR), Prefix Contrastive Loss (PCL), and Context Diversification via Transposition (CDT).

The baseline (top row) corresponds to a simplified version of our model that uses only entity annotation for input passages and queries, without any of the proposed learning objectives or data augmentation techniques. This setup resembles prior generative retrieval approaches such as DSI-QG or NCI (with BGE-Reranker), but with annotated inputs.

Specifically, enabling each individual component leads to consistent performance improvements over the baseline. In particular, adding EMR alone improves Hits@1 by +0.64 on PAIR and +0.58 on LiHuaWorld, suggesting that encouraging the model to reduce over-reliance on ambiguous entities leads to more robust representations. Similarly, introducing PCL boosts MRR@10 by +0.62 on PAIR, highlighting its role in aligning structurally similar but semantically distinct queries and passages. CDT further contributes by enhancing generalization across contexts; its standalone impact is especially visible in Hits@1 and Hits@10.

Combining any two components generally improves retrieval performance, but the full model — with all three components jointly trained — achieves the highest scores across all metrics on both datasets. On PAIR, the complete model improves Hits@1 from 59.84 (baseline) to 61.15 and MRR@10 from 67.06 to 68.37. On LiHuaWorld, the gains are even more pronounced, where the full model achieves 50.19 Hits@1 and 55.31 MRR@10 — outperforming all reduced variants.

These results confirm that each component of PEARL contributes uniquely and synergistically to the overall retrieval performance. Entity-Mass Regularization stabilizes focus on structural cues, Prefix Contrastive Loss improves discriminability in embedding space, and Context Diversification enhances the model’s generalization

Table 4: Ablation study of PEARL on PAIR and LiHuaWorld. We evaluate the impact of each core component: Entity-Mass Regularization (EMR), Prefix Contrastive Loss (PCL), and Context Diversification via Transposition (CDT). Removing any component results in performance degradation across all metrics, confirming that each module contributes meaningfully to the overall performance of PEARL.

	EMR	PCL	CDT	PAIR				LiHuaWorld [11]			
				H@1	H@5	H@10	M@10	H@1	H@5	H@10	M@10
	$\times$	$\times$	$\times$	59.84	76.57	80.30	67.06	48.57	58.69	61.22	53.02
	✓	✓	$\times$	60.30	76.73	80.63	67.38	48.93	59.21	61.35	53.39
	✓	$\times$	✓	60.69	77.12	80.61	67.75	49.73	60.76	62.32	54.21
	$\times$	✓	✓	60.51	77.20	80.56	67.61	49.51	59.80	61.44	54.01
PEARL (proposed)				61.15	77.85	81.46	68.37	50.19	61.87	63.23	55.31

to unseen phrasings. Removing any module leads to notable performance degradation, affirming the necessity of a unified design for robust personalized retrieval.

4.4 Hyperparameter Sensitivity Analysis (RQ3)

To assess the robustness of PEARL with respect to its key hyperparameters, we conduct a sensitivity analysis on the PAIR dataset. Specifically, we examine the effects of four major hyperparameters introduced in our framework: (a) the threshold δ for entity-span mass regularization (Section 3.1), (b) the weighting factor γ for entity-structural embeddings (Section 3.2), (c) the loss weight λ_1 for the attention-based entity regularization term $\mathcal{L}_1$, and (d) the loss weight λ_2 for the prefix contrastive loss $\mathcal{L}_2$.

Figure 4 shows Hits@1 and Hits@5 across varying hyperparameter values. Performance remains stable within reasonable ranges, indicating low sensitivity to specific settings. The model performs consistently well around $\delta = 5e-3$, $\gamma = 0.5$, and $\lambda_1 = \lambda_2 = 1.0$, which we use as defaults. Minor variations in performance suggest that while each component contributes, the overall training scheme is robust across configurations.

5 Related Work

Generative Retrieval Paradigms. Generative retrieval formulates retrieval as a sequence-to-sequence task, allowing a unified model to perform both indexing and lookup. *Differentiable Search Index (DSI)* [33], one of the earliest models, maps documents to tokenized identifiers (e.g., document IDs) and learns end-to-end retrieval using only transformer parameters. *Neural Corpus Indexer (NCI)* [35] and *DSI-QG* [40] further bridges the gap between indexing and retrieval by representing each document through multiple synthetic queries, aligning training closer to actual retrieval usage, while NCI incorporates prefix-tuned decoding to produce semantically meaningful identifiers. *MINDER* [22] enriches semantic alignment using multi-view identifiers, assigning each document titles, n-grams, and pseudo-queries. Other works, such as [31], optimize ID tokenization to reduce generation ambiguity. Despite progress, generative retrievers remain content-agnostic and struggle with entity ambiguity — especially in personalized contexts, where the same mention may refer to different entities. We address this by incorporating Named-Entity Recognition (NER) during both indexing and retrieval.

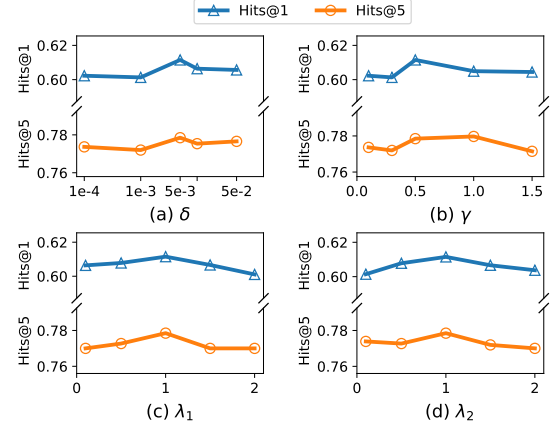


Figure 4: Sensitivity of PEARL to key hyperparameters on the PAIR dataset. We report Hits@1 and Hits@5 for (a) entity-span mass threshold δ , (b) entity-structural embedding weight γ , (c) entity-mass regularization loss weight λ_1 , and (d) prefix contrastive loss weight λ_2 (details in Section 4.4)

Entity Disambiguation in Retrieval. Entity ambiguity is a long-standing challenge in information retrieval. The Amber benchmark [5] shows that retrievers often exhibit popularity bias, favoring common entities when disambiguation cues are lacking. Traditional approaches have used NER and entity linking as pre-processing or auxiliary signals. Hasibi et al. [13] showed that annotating queries with linked entities improves ranking via feature augmentation. More recently, entity-aware sparse retrievers have emerged: Shehata et al. [30] enhance queries and documents with entity-linked expansions to boost recall, while De Cao et al. [9] use NER-derived context and type constraints to resolve ambiguity. However, most remain feature-based or pipelined. In contrast, our approach integrates NER into the generative retriever’s training loop, enabling context-aware indexing that combines the adaptability of generative models with reduced reliance on surface-form memorization.

6 Conclusion

In this paper, we propose PEARL, a novel entity-aware generative retrieval framework for personalized information retrieval. To address ambiguous, user-specific references in queries, PEARL integrates three key techniques: entity-span mass regularization to reduce lexical sensitivity, prefix-based contrastive learning for structural alignment, and context diversification via transposition to improve robustness. These components are jointly trained under a unified sequence-to-sequence objective using semantically clustered identifiers derived from hierarchical k -means. Experiments on public and synthetic PIR datasets show that PEARL consistently outperforms sparse, dense, and generative baselines across all zero-shot retrieval metrics. Future work will explore continual user adaptation, scalable identifier construction, and cross-modal retrieval.

Acknowledgments

This was supported by Mobile eXperience(MX) Business, Samsung Electronics Co., Ltd.

A GenAI Usage Disclosure

In this work, we synthetically generated the proposed dataset PAIR using large language models. Specifically, we used GPT-4o [1] from OpenAI via API calls for persona generation, and Qwen2.5-14B-Instruct-1M [38] from Qwen for large-scale query-passage pair generation, running locally on 4× A100-80GB GPUs.

The user profiles — personas — used to contextualize the query-passage pairs were generated via dedicated prompts designed to synthesize fictional users with realistic names, occupations, hobbies, frequent locations, and interpersonal relationships.

Subsequently, query-passage pairs were created using three distinct prompt templates tailored to specific data types: (1) calendar events, (2) messages, and (3) contact entries. These prompts are conditioned on the generated user profiles, and include detailed instructions and examples to guide the models in producing realistic, contextually grounded, and diverse outputs.

Examples of the prompts used during the generation process are provided in Sections A.1, A.2, A.3, and A.4.

Followed by the generation process, all samples underwent a multi-stage, rule-based post-processing pipeline to ensure temporal consistency, contextual accuracy, and semantic diversity. Key steps include:

Date Consistency Adjustments. Many queries include temporal expressions (e.g., "this weekend", "last month"), and the corresponding passage timestamps were programmatically adjusted to reflect these references accurately. For example, if a query referenced "this weekend" but the associated calendar event did not fall on the correct Saturday or Sunday, the passage was manually corrected.

Query Cleanup and Filtering. Certain query patterns that were incoherent or semantically incompatible with their associated passages were either removed or rephrased. This includes queries such as "Who did I meet next weekend?" or "Did I go next month?".

Deduplication. To reduce redundancy, samples were deduplicated in two phases. First, embedding-based deduplication was applied using TF-IDF and cosine similarity across queries and passage texts. Second, n-gram-based deduplication was performed by filtering out queries with overlapping tri-gram patterns after stopword removal and normalization.

Year Balancing. The LLMs exhibited a strong bias toward generating events dated in 2024. To mitigate this, the pipeline imposed a configurable cap on 2024-dated samples. When the cap is exceeded, timestamps are programmatically shifted to earlier years (e.g., 2018–2023) while preserving contextual consistency.

A.1 User Profile Generation Prompt

The following is a list of structured persona entries. Each entry contains rich biographical details, hobbies, relationships, and locations.

Generate (N) new personas that match the tone, structure, and formatting style of the existing entries, but have distinct personalities, occupations, hobbies, and relationships.

Each generated persona must satisfy the following rules:

- Must include exactly one mother and one father relationship (with appropriate names and brief details).
- Include at least:
 - 5 hobbies
 - 5 frequent locations
 - 8 relationships
- At least 5 relationships must include **nicknames**, using either:
 - Nicknames not derived from names (e.g., *jellybean*, *darling*, *chief*, *my love*, *champ*, etc.)
 - Abbreviated forms of names (e.g., *Stephen* → *Steve*, *Katherine* → *Kathy*)
- Relationships should be diverse (family, friends, colleagues, therapist, etc.) and described with naturalistic, specific traits or interactions.
- **Do not reuse hobbies** that have already been generated in the previous personas. Avoid common hobbies such as hiking, practicing yoga, and attending art exhibitions.
- **Do not reuse names** that have already been included in previous personas.
- There should be at least:
 - 1 teenager
 - 1 young adult
 - 1 middle-aged adult
 - 1 elder persona

These persona entries that have already been generated:
(... previously generated user profiles ...)

Now continue the list by adding new personas.

A.2 Calendar Event Type Generation Prompt

The following text is a simplified version of the prompt that was used. Please refer to our GitHub page for the full prompt.

Generate exactly (N) unique query-passage pairs grounded in a user's personal context. The passages should resemble data from a user's calendar events.

Important Requirements:

- Queries should refer to calendar events or reminders in a way that assumes the user remembers partial or vague details.
- **Do not** use any noun phrase, keyword, or core term from the query in the corresponding passage. Use indirect clues, context associations, or paraphrasing. Example:

Query

- TXT: Did I attend my cooking thing?
- CT: 2021-10-18T18:29:20

Passage

- TITLE: Culinary Course
- DESC: Learning to make chicken pot pie.
- DATE: 2021-09-02T14:30:00
- CT: 2024-08-01T21:30:17
- LOCATION: Queens Culinary Academy
- ATTENDEES: Chef Rina

This is a **good** example as the passage paraphrases "cooking".

- Use a variety of time-related expressions across the dataset.
 - For a quarter of the passages, use **explicit** time expressions such as "next spring", "2 weeks ago", or "on the last day of May".
 - For another quarter, use **implicit** time expressions such as "after that party last weekend", "before class", or "after the graduation ceremony".
- **Do not** generate query-passage pairs that create one-to-many relationships. The goal is for every query to be **uniquely answerable** by one-and only one-passage, with no ambiguity or overlap across the dataset.
- Use ISO 8601 format (YYYY-MM-DDTHH:MM:SS) for all CT and DATE fields. The query CT (creation time) represents when the query was created, and the passage CT represents when the calendar event was created. The passage DATE field represents the actual event time.
- **Do not** generate queries that contain verbs like "mention", "talk about", "remind", "text", or other communication-related actions. Queries with such words must be answered with message-type passages, not calendar events.
- Reference the user profile to generate queries and passages grounded in personal context. Incorporate details from the user's profile such as: occupation, hobbies, frequently visited locations, and close relationships.
- In at least half of the query-passage pairs, use the nicknames specified in the user profile.
- ...

A.3 Message Type Generation Prompt

The following is a simplified version of the original prompt, with overlapping instructions from Section A.2 omitted. Please refer to our GitHub page for the full prompt.

Generate exactly (N) unique query-passage pairs grounded in a user's personal context. The passages should resemble data from a user's messages.

Important Requirements:

- **Do not** repeat queries from previous generations.
- Use ISO 8601 format (YYYY-MM-DDTHH:MM:SS) for all CT fields. The query CT (creation time) represents when the query was created, and the passage CT represents when the message was sent.
- If the sender is a business or facility, it should be sending formal messages like event notifications, reminders, or appointment details. **Do not** write messages where a business or facility talks about personal or social interactions (e.g., "hung out," "caught up," "chatted," "enjoyed", etc.). Example:

Query

- TXT: Who is the person I met at the local coffee shop?
- CT: 2025-10-15T14:45:00

Passage

- FROM: Beanstalk Brewery
- RELATION: Regular Spot
- GROUP: Coffee Places
- CT: 2025-06-10T11:30:00
- BODY: Enjoyed catching up with a fellow brew aficionado.
- DATE: 2025-09-18T15:30:00

This is a **bad** example because "Beanstalk Brewery" is a coffee shop, but the message BODY field implies that the business had a personal interaction with the user.

- ...

A.4 Contact Type Generation Prompt

The following is a simplified version of the original prompt, with overlapping instructions from Section A.3 omitted. Please refer to our GitHub page for the full prompt.

Generate exactly (N) unique query-passage pairs grounded in a user's personal context. The passages should resemble data from a user's contacts.

Important Requirements:

- **Do not** repeat queries from previous generations.
- Generate queries that ask directly about a contact's details—such as name, email, date added, or relationship. Avoid queries about conversations or reminders. Answers must come from the contact entry alone, not message history.
- Use ISO 8601 format (YYYY-MM-DDTHH:MM:SS) for all CT fields. The query CT (creation time) represents when the query was created, and the passage CT represents when the contact entry was created.
- The names in the NAME field must always be actual names, not nicknames. Example:

Query

- TXT: What is the email address of my friend Nerd who works as a detective?
- CT: 2021-11-03T16:45:00

Passage

- NAME: Connor
- RELATION: Friend
- GROUP: Police Dept. Contacts
- EMAIL: connor@example.net
- DATE: 2020-07-25T17:20:00

This is a **good** example because although the query refers to the person by their nickname, "Nerd", the NAME field includes the actual name "Connor" and not the nickname.

• ...

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Alan Akbik, Tanja Bergmann, Duncan Blythe, Kashif Rasul, Stefan Schweter, and Roland Vollgraf. 2019. FLAIR: An easy-to-use framework for state-of-the-art NLP. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics (demonstrations)*. 54–59.
- [3] Alan Akbik, Duncan Blythe, and Roland Vollgraf. 2018. Contextual string embeddings for sequence labeling. In *Proceedings of the 27th international conference on computational linguistics*. 1638–1649.
- [4] Seyed Ali Bahrainian and Fabio Crestani. 2017. Towards the next generation of personal assistants: systems that know when you forget. In *Proceedings of the ACM SIGIR International Conference on Theory of Information Retrieval*. 169–176.
- [5] Anthony Chen, Pallavi Gudipati, Shayne Longpre, Xiao Ling, and Sameer Singh. 2021. Evaluating entity disambiguation and the role of popularity in retrieval-based NLP. *arXiv preprint arXiv:2106.06830* (2021).
- [6] Haonan Chen, Zhicheng Dou, Kelong Mao, Jiongnan Liu, and Ziliang Zhao. 2024. Generalizing conversational dense retrieval via llm-cognition data augmentation. *arXiv preprint arXiv:2402.07092* (2024).
- [7] Jiangui Chen, Ruqing Zhang, Jiafeng Guo, Maarten de Rijke, Wei Chen, Yixing Fan, and Xueqi Cheng. 2023. Continual learning for generative retrieval over dynamic corpora. In *Proceedings of the 32nd ACM international conference on information and knowledge management*. 306–315.
- [8] Xilun Chen, Kushal Lakhotia, Barlas Oguz, Anchit Gupta, Patrick Lewis, Stan Peshterliev, Yashar Mehdad, Sonal Gupta, and Wen-tau Yih. 2021. Salient phrase aware dense retrieval: Can a dense retriever imitate a sparse one? *arXiv preprint arXiv:2110.06918* (2021).
- [9] Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. 2020. Autoregressive entity retrieval. *arXiv preprint arXiv:2010.00904* (2020).
- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.
- [11] Tianyu Fan, Jingyuan Wang, Xubin Ren, and Chao Huang. 2025. MiniRAG: Towards Extremely Simple Retrieval-Augmented Generation. *arXiv preprint arXiv:2501.06713* (2025).
- [12] Shuai Guo, Yang Gu, Yuan Ma, Yingwei Zhang, Weining Weng, Jun Liu, Weiwei Dai, and Yiqiang Chen. 2024. Information Retrieval Optimization for Non-Exemplar Class Incremental Learning. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 717–726.
- [13] Faegheh Hasibi, Krisztian Balog, and Svein Erik Bratsberg. 2016. Exploiting entity linking in queries for entity retrieval. In *Proceedings of the 2016 acm international conference on the theory of information retrieval*. 209–218.
- [14] Hyunsik Jeon, Jongjin Kim, Jaeri Lee, Jong-eun Lee, and U Kang. 2023. Aggregately Diversified Bundle Recommendation via Popularity Debiasing and Configuration-Aware Reranking. In *Advances in Knowledge Discovery and Data Mining - 27th Pacific-Asia Conference on Knowledge Discovery and Data Mining, PAKDD 2023, Osaka, Japan, May 25-28, 2023, Proceedings, Part III (Lecture Notes in Computer Science, Vol. 13937)*, Hisashi Kashima, Tsuyoshi Idé, and Wen-Chih Peng (Eds.). Springer, 348–360.
- [15] Woojeon Jin, Jinhong Jung, and U. Kang. 2019. Supervised and extended restart in random walks for ranking and link prediction in networks. *PLOS ONE* 14, 3 (03 2019), 1–23.
- [16] Jinhong Jung, Woojeon Jin, and U Kang. 2020. Random walk-based ranking in signed social networks: model and algorithms. *Knowl. Inf. Syst.* 62, 2 (2020), 571–610. doi:10.1007/S10115-019-01364-Z
- [17] Jinhong Jung, Woojeon Jin, Lee Sael, and U Kang. 2016. Personalized ranking in signed networks using signed random walk with restart. In *2016 IEEE 16th international conference on data mining (ICDM)*. IEEE, 973–978.
- [18] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. [n.d.]. Dense Passage Retrieval for Open-Domain Question Answering.
- [19] Mei Kobayashi and Koichi Takeda. 2000. Information retrieval on the web. *ACM computing surveys (CSUR)* 32, 2 (2000), 144–173.
- [20] Bonhun Koo, Hyunsik Jeon, and U Kang. 2020. Accurate news recommendation coalescing personal and global temporal preferences. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 78–90.
- [21] Bonhun Koo, Hyunsik Jeon, and U Kang. 2021. PGT: news recommendation coalescing personal and global temporal preferences. *Knowl. Inf. Syst.* 63, 12 (2021), 3139–3158.
- [22] Yongqi Li, Nan Yang, Liang Wang, Furu Wei, and Wenjie Li. 2023. Multiview identifiers enhanced generative retrieval. *arXiv preprint arXiv:2305.16675* (2023).
- [23] Antonio Mallia, Omar Khattab, Torsten Suel, and Nicola Tonellotto. 2021. Learning passage impacts for inverted indexes. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1723–1727.
- [24] Jianmo Ni, Gustavo Hernandez Abrego, Noah Constant, Ji Ma, Keith B Hall, Daniel Cer, and Yinfei Yang. 2021. Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models. *arXiv preprint arXiv:2108.08877* (2021).
- [25] Jianmo Ni, Chen Qu, Jing Lu, Zhu Yun Dai, Gustavo Hernández Ábrego, Ji Ma, Vincent Y Zhao, Yi Luan, Keith B Hall, Ming-Wei Chang, et al. 2021. Large dual encoders are generalizable retrievers. *arXiv preprint arXiv:2112.07899* (2021).
- [26] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).
- [27] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.
- [28] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 10299–10315.
- [29] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.
- [30] Dahlia Shehata, Negar Arabzadeh, and Charles LA Clarke. 2022. Early stage sparse retrieval with entity linking. In *Proceedings of the 31st acm international conference on information & knowledge management*. 4464–4469.
- [31] Weiwei Sun, Lingyong Yan, Zheng Chen, Shuaiqiang Wang, Haichao Zhu, Pengjie Ren, Zhumin Chen, Dawei Yin, Maarten Rijke, and Zhaochun Ren. 2023. Learning to tokenize for generative retrieval. *Advances in Neural Information Processing*

- Systems* 36 (2023), 46345–46361.
- [32] Qiaoyu Tang, Jiawei Chen, Zhuoqun Li, Bowen Yu, Yaojie Lu, Haiyang Yu, Hongyu Lin, Fei Huang, Ben He, Xianpei Han, et al. 2024. Self-retrieval: End-to-end information retrieval with one large language model. *Advances in Neural Information Processing Systems* 37 (2024), 63510–63533.
 - [33] Yi Tay, Vinh Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Gupta, et al. 2022. Transformer memory as a differentiable search index. *Advances in Neural Information Processing Systems* 35 (2022), 21831–21843.
 - [34] Ghazaleh H Torbati, Andrew Yates, and Gerhard Weikum. 2020. Personalized entity search by sparse and scrutable user profiles. In *Proceedings of the 2020 Conference on Human Information Interaction and Retrieval*. 427–431.
 - [35] Yujing Wang, Yingyan Hou, Haonan Wang, Ziming Miao, Shibin Wu, Qi Chen, Yuqing Xia, Chengmin Chi, Guoshuai Zhao, Zheng Liu, et al. 2022. A neural corpus indexer for document retrieval. *Advances in Neural Information Processing Systems* 35 (2022), 25600–25614.
 - [36] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 38–45.
 - [37] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2023. C-pack: packaged resources to advance general Chinese embedding. 2023. *arXiv preprint arXiv:2309.07597* (2023).
 - [38] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115* (2024).
 - [39] Ziliang Zhao, Zhicheng Dou, and Yujia Zhou. 2024. Generating Intent-aware Clarifying Questions in Conversational Information Retrieval Systems. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 3384–3394.
 - [40] Shengyao Zhuang, Houxing Ren, Linjun Shou, Jian Pei, Ming Gong, Guido Zuccon, and Daxin Jiang. 2022. Bridging the gap between indexing and retrieval for differentiable search index with query generation. *arXiv preprint arXiv:2206.10128* (2022).